

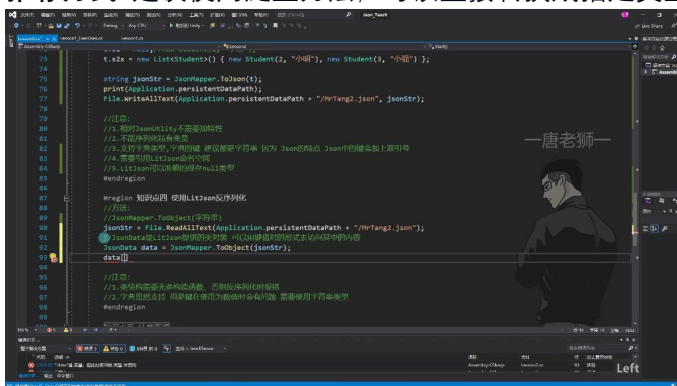
一、文件序列化和反序列化 00:04

1. 上一节回顾

2. 使用LitJson进行反序列化 00:27

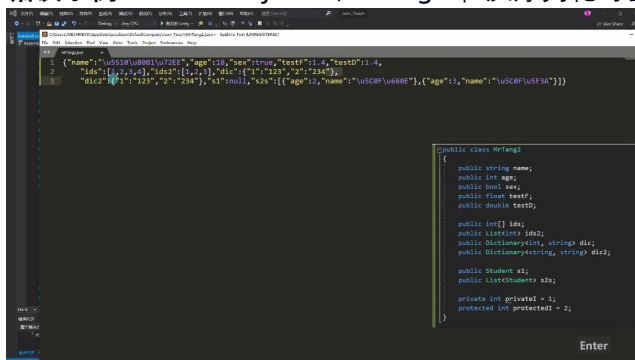
1) 基本方法

- **核心方法:** 使用 `JsonMapper.ToObject()` 方法, 将JSON字符串转换为对象
- **两种重载:**
 - 直接返回 `JsonData` 对象: `JsonMapper.ToObject(jsonStr)`
 - 泛型方法返回指定类型: `JsonMapper.ToObject<T>(jsonStr)`
- **推荐方式:** 建议使用泛型方法, 可以直接转换成指定类型, 使用更方便



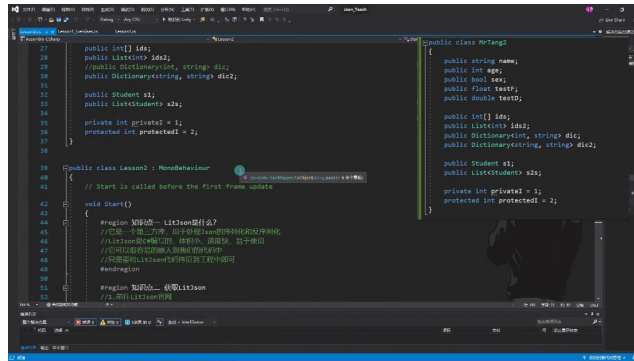
2) 注意事项 04:33

- **字典键类型限制**
 - **问题原因:** JSON规范要求键必须加双引号, 导致数值型键会被自动转为字符串
 - **解决方案:** 字典键建议统一使用字符串类型
 - **错误示例:** `Dictionary<int, string>` 在反序列化时会报类型转换错误



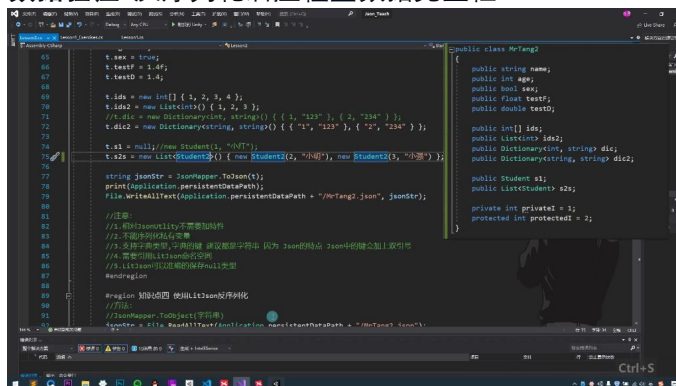
● 无参构造函数要求 06:19

- **必要条件:** 反序列化的类必须有无参构造函数
- **常见错误:** 自定义有参构造函数会覆盖默认无参构造
- **解决方法:** 显式添加无参构造函数, 即使内容为空



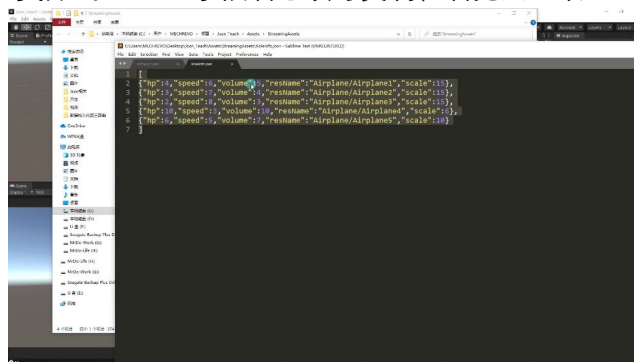
3) 实际应用 08:43

- 文件读取: 使用File.ReadAllText()读取JSON文件
- 类型转换: 通过泛型方法直接转换为目标类型
- 数据验证: 反序列化后应检查数据完整性



4) 高级特性 10:17

- 直接读取数据集 10:43
 - 优势: 相比JsonUtility, LitJson可以直接读取数组和集合
 - 支持类型: 数组(T[])和列表(List<T>)
 - 字段匹配: JSON字段名必须与类属性名完全一致

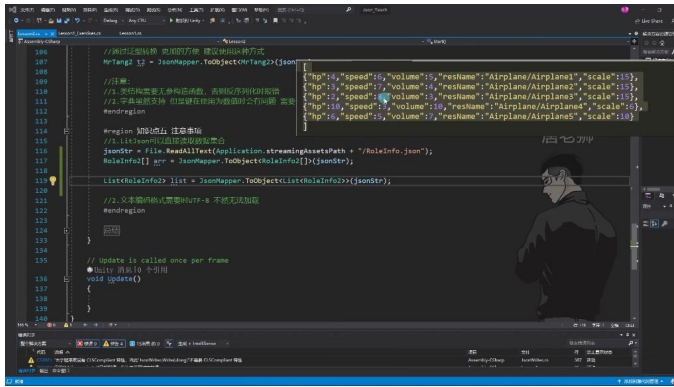


● 编码格式要求 13:18

- 编码格式: 必须使用UTF-8编码, 否则无法正确加载
- 文件检查: 确保JSON文件保存为UTF-8格式

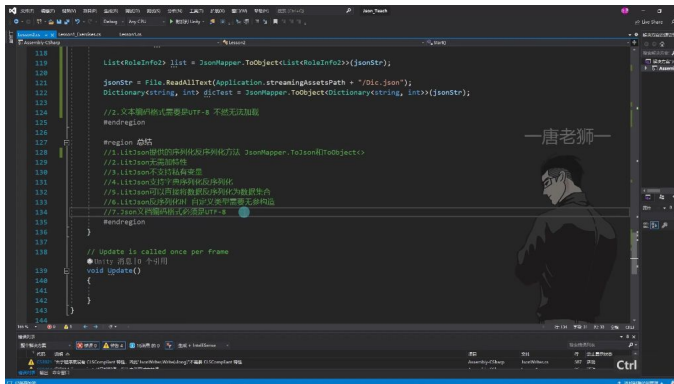
5) 总结 14:27

- 核心方法: JsonSerializer.ToObject<T>()是最常用的反序列化方式
- 两大坑点:
 - 字典键必须使用字符串类型
 - 类必须有无参构造函数
- 优势特性: 支持直接读取数组/集合, 比JsonUtility更灵活



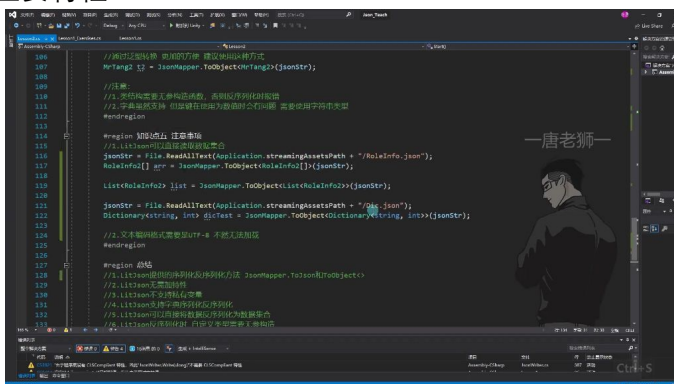
3. LitJson知识点总结

1) 核心方法



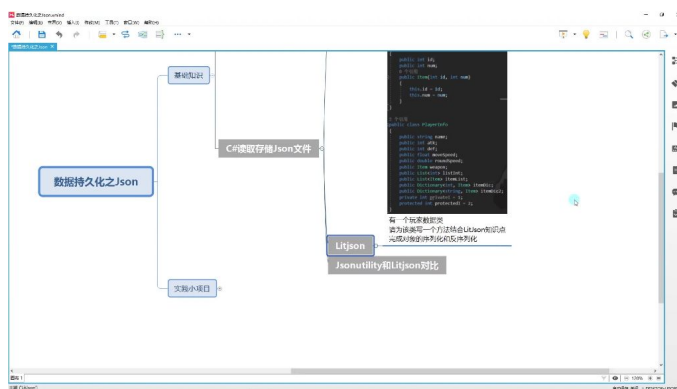
- 序列化方法: `JsonMapper.ToJson()`, 用于将对象转换为JSON字符串
- 反序列化方法: `JsonMapper.ToObject<T>()`, 建议使用泛型重载更方便
- 集合处理: 可以直接将数据反序列化为数组或List集合

2) 主要特性

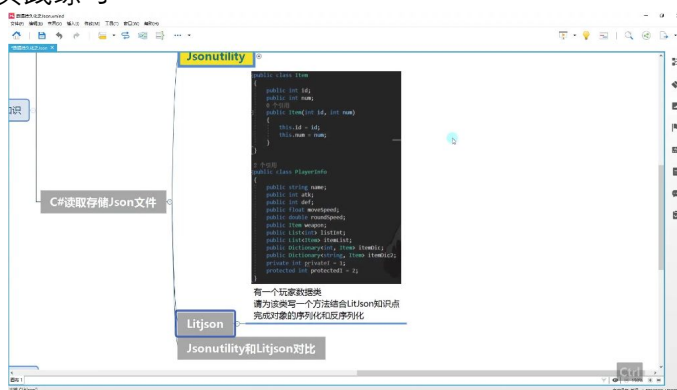


- 特性支持: 无需添加任何特性即可序列化/反序列化
- 变量限制: 不支持私有变量的序列化, 与JsonUtility的主要区别
- 字典支持: 支持字典的序列化/反序列化, 但键必须是字符串类型
- 编码要求: JSON文档编码格式必须是UTF-8, 否则无法加载

3) 注意事项



- - 构造函数要求: 自定义类型必须有无参构造函数, 否则反序列化会报错
 - 集合处理: 可以直接将JSON数据反序列化为数据集合 (数组或List)
 - 类型转换: 通过泛型转换更加方便, 建议使用泛型方式
 - 字典限制: 虽然支持字典, 但键为数值类型时会有问题, 必须使用字符串类型键
- 4) 实践练习



-
- 练习要求: 为PlayerInfo类编写方法, 使用LitJson完成对象的序列化和反序列化
- 对比实践: 需要比较JsonUtility和LitJson在实际使用中的区别

二、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
JSON反序列化方法	使用 JsonMapper.ToObject<T>泛型方法, 传入JSON字符串转换为指定类型对象	必须使用泛型方法, 非泛型方法返回JsonData类型需通过键值对访问	★★
字典反序列化限制	仅支持键为字符串类型的字典, 数值类型键会因JSON格式强制转为字符串导致报错	键必须为 string , 否则反序列化失败	★★★
无参构造函数要求	自定义类需保留无参构造函数, 若定义有参构造需显式补充无参构造	无参构造被覆盖会报错, 如Student2类需手动添加	★★★
数据集合支持	可直接反序列化为数组或List<T>, 无需像JsonUtility额外包裹类	直接读取数组/List, 如RowingInfo[]或List<RowingInfo>	★

编码 格式 要求	JSON文件必须为UTF-8编码， 否则无法加载	与JsonUtility一致，非UTF-8会报 错	★
私有 变量 限制	不支持私有变量序列化/反序 列化，与JsonUtility不同	仅支持公共字段/属性	★ ★
JSON 语法 校验	反序列化时需严格校验JSON 格式（如末尾不能有多余逗 号）	格式错误直接报错，需人工检查 文件	★ ★ ★